World Action Models for Cross Embodiment Robot Control

D1 Detailed Design | Joshua Jerin and Krrish Thakku Suresh
Advisor: Tri Nguyen | CS 5001 |

1 Header, scope, and conventions

Our goal is to adapt a pretrained world action model to a new robot arm with a small set of demonstrations, compare its performance across robots, and test it safely in a controlled lab setting.

Scope. This document details C2 Model, including the experiment registry that holds the project's data model, and C4 Motion safety, whose per-step command check and hardware gate break the project if they are slow or wrong; C1 Experiment setup, C3 Simulation and evaluation, and C5 Robot interface are deferred to D2, although Figure 1 already shows the registry tables they will write.

Conventions. Figure 1 (data model): each box is an entity stored as one table; the header names the entity and, in parentheses, the component that writes it (colour repeats this but carries no extra meaning). Underlined attributes marked PK form the primary key; italic attributes marked FK refer to the entity at the other end of a line. Each line is a relationship, and the numbers at each end give the cardinality at that end as minimum..maximum: 1 means exactly one, 0..1 at most one, 0..* zero or more, 1..* one or more. Figure 2 (algorithm): rounded boxes are processing steps inside C4, dashed boxes are data entering or leaving C4, the octagon stops the trial, the plain box rejects a single proposal, the cylinder is a registry write, solid arrows are control flow labelled with the check result, and the dotted arrow is an asynchronous write. Both figures carry their own legend. Interface IDs I1 to I10 and component IDs C1 to C5 are unchanged from D0, and US-01 to US-05 are the submitted user stories.

Changes since D0. D0 stored every output as an ordinary project file. D1 keeps those files but adds one SQLite registry file that indexes them, so C4 can verify which data, model and report belong together (Section 2). SQLite is a file, not a service, so D0's decision against a separate storage service still holds. The I5 test report remains a JSON file; the registry stores its path and hash. I4, I7 and I9 messages gain an `api_version` field and named error codes (Section 5).

Component and interface reference (from D0)

ID	Component	Responsibility
C1	Experiment setup	Validates the researcher's dataset and robot setup before an experiment starts
C2	Model	Adapts the pretrained model and predicts robot actions
C3	Simulation and evaluation	Tests the model in simulation and compares it with a single-embodiment baseline
C4	Motion safety	Approves physical trials and keeps every command within the approved limits
C5	Robot interface	Passes sensor data in and checked commands out; the only component that talks to the robot

ID	Connects	Carries
I1	Researcher ↔ C1	Setup request and validation result
I2	C1 ↔ C2	Validated demonstrations and settings; saved model and report paths
I3	C2 ↔ C3	Checkpoint for simulation testing; simulated observations and actions

ID	Connects	Carries
I4	C2 ↔ C4	Predicted joint velocities from C2; run permission or stop from C4
I5	C3 → C4	Simulation test report
I6	C5 → C2	Camera image and joint readings
I7	C4 ↔ C5	Checked commands and heartbeat to C5; robot state and acknowledgement back
I8	C5 ↔ robot	Device messages to and from the hardware
I9	Researcher ↔ C4	Trial start or stop request and its result
I10	Researcher → robot	Physical emergency stop, independent of the software

2 Data model, the D1 diagram

The data model belongs to C2's experiment registry, which records provenance for every artifact the project produces. We chose a **relational model in SQLite**. The hardware gate (Section 3.2) must join a checkpoint to its evaluation run, the run's episodes and the robot, and US-01 requires that every checkpoint can be traced to the exact demonstrations, robot description, settings and code that produced it. Foreign keys and unique constraints enforce those links at write time. SQLite runs in-process from Python's standard library with no server, so it adds no deployment work. Large data (demonstration arrays, images, model weights) stays in files on disk; rows hold each file's path and SHA-256 hash so a swapped or edited file is detected. We rejected keeping only per-run JSON files, as D0 implied, because nothing would check that files refer to each other consistently or stop a duplicated episode from inflating a success rate.

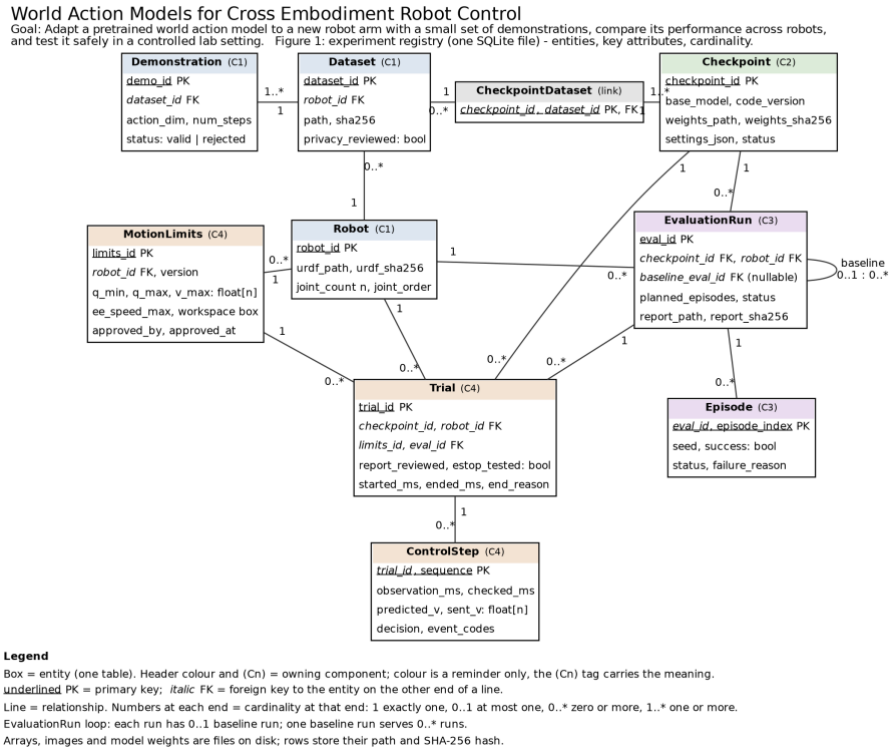


Figure 1. Experiment registry: entities, key attributes and cardinality. Source: Design_Diagrams/fig1_data_model.dot.

Structural decisions

A **checkpoint** is a saved copy of the model at a particular moment: a snapshot of everything it has learned.
A **demonstration** is one recorded example of the robot doing the task correctly, used to teach the model.
An **episode** is one complete attempt at the task, from start to finish, in simulation.
An **evaluation run** is one full simulation test of a model
A **control step** is one tiny moment of a real-robot trial

Decision	Reason	Req.
Demonstration is an entity, not a count on Dataset.	C1 validates each demonstration separately and may reject one for a wrong action dimension or exclude one for privacy. Per-demonstration status records exactly which demonstrations trained a checkpoint.	US-01, US-05, I2
MotionLimits is a separate, versioned entity rather than columns on Robot.	Limits change after advisor review. Each Trial must point to the exact approved version used. Rows are never edited; a change inserts a new version.	US-03, US-04, I9
Checkpoint to Dataset is many-to-many through CheckpointDataset.	A cross-embodiment checkpoint is trained on datasets from several robots, and one dataset is reused by many checkpoints. The checkpoint end is 1..* because an adapted checkpoint must cite at least one dataset. The pretrained base model is stored as the base_model string, not a Checkpoint row, because we did not produce it.	US-01, US-02
Whether a robot was seen in training is derived, not stored.	A robot is seen for a checkpoint when it owns one of that checkpoint's datasets. A stored flag could go stale when datasets are added; a join cannot.	US-02
Episode is an entity; success rate is computed, not stored.	The gate recounts episodes itself, so a hand-edited summary cannot pass. The composite key (eval_id, episode_index) makes a duplicated episode impossible.	US-04, I5
EvaluationRun has an optional self-reference to its baseline (0..1 to 0..*).	Each candidate run points to the single-embodiment baseline run on the same robot and seeds; one baseline run serves many candidates. Baseline runs have no baseline, so the key is nullable.	US-02
Trial references the EvaluationRun that authorized it (1 to 0..*).	Answers the question of which simulation evidence allowed a given physical run.	US-04, I5, I9
ControlStep key is (trial_id, sequence).	One row per control step. The key makes a replayed sequence number impossible to log twice, and the 100-step target is a simple count.	US-03, US-04, I7
Each component writes only its own tables.	Tagged (C1) to (C4) in Figure 1. Other components read but never write them, which keeps ownership as clear as in D0.	D0 Sec. 3

Indexes

Index	Why
Checkpoint.weights_sha256 (unique)	Registering identical weights twice is a duplicate; C2 also looks a checkpoint up by hash when loading weights.
EvaluationRun (checkpoint_id, robot_id, status)	The gate's main lookup: the completed run for this checkpoint on this robot.
Episode (eval_id, episode_index), primary key	The gate counts and sums episodes for one run; the key also blocks duplicates.
MotionLimits (robot_id, version) (unique)	Finds the latest approved limits for a robot; prevents two rows claiming the same version.

Index	Why
ControlStep (trial_id, sequence), primary key	Reads a trial's log in order; rows are append-only.
Dataset (robot_id)	The seen-or-unseen join for US-02.
Not indexed: ControlStep.decision, Episode.success	Scans of one trial or one run touch at most a few thousand rows; an index would only slow inserts.

Size. Realistic: about 1-2 robots, 10 datasets, 500 demonstrations, 30 checkpoints, 60 evaluation runs (3,000 episodes) and 20 trials of up to 2,000 steps each (40,000 ControlStep rows, a few megabytes). At 100 times that, ControlStep reaches about 4 million rows, roughly 1 to 2 GB, which SQLite still handles; the write rate (20 rows per second during a trial) does not change, so scale does not affect the control loop.

3 Core algorithms

Two computations in C4 break the project if they are slow or wrong: the per-step command check, which runs in every 50 ms control cycle, and the hardware gate, which decides whether a physical trial may start at all.

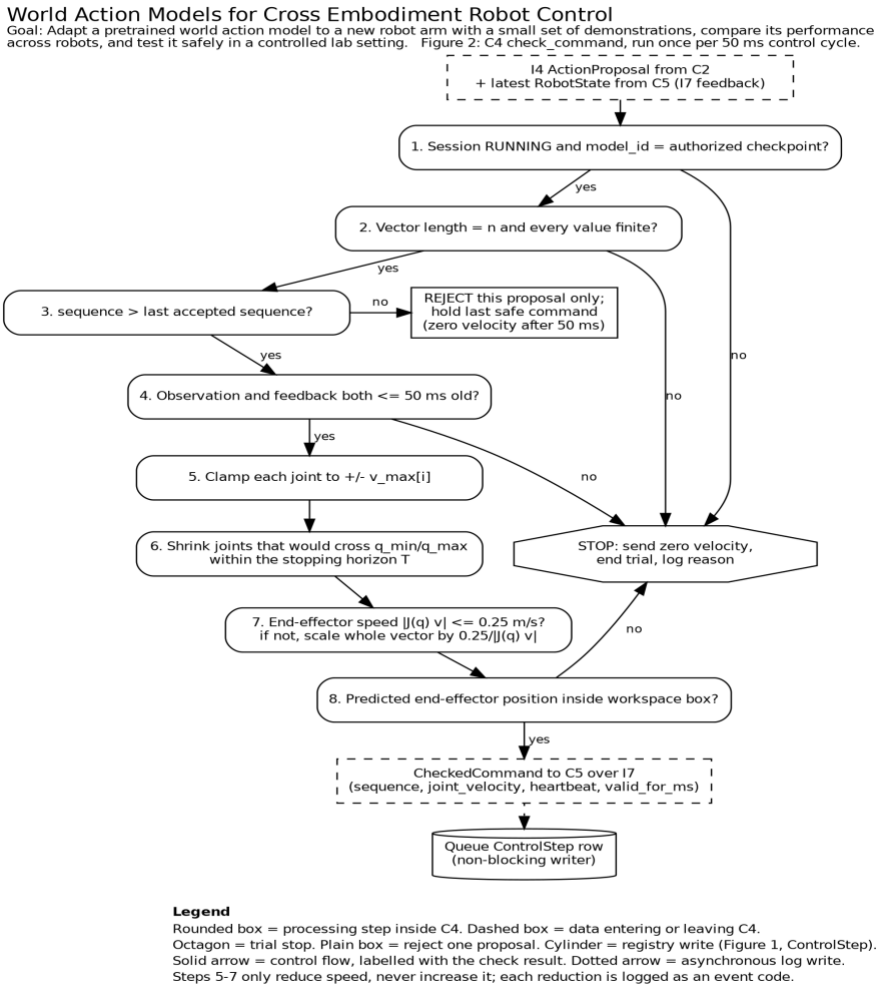
3.1 check_command (C4, every control cycle)

What it is and the problem it solves. check_command turns each action proposed by the model into a command that is guaranteed to stay within the approved limits, or it stops the trial. If it is wrong, the arm can exceed the 0.25 m/s end-effector limit or leave the workspace (US-03, US-04). If it is slow, it eats into the 50 ms cycle (budget 5 ms), commands arrive stale and every trial stops.

Inputs and outputs. All times are integer milliseconds from the same computer's monotonic clock (CLOCK_MONOTONIC); n is the registered joint count; vectors are float64 in the robot's registered joint order.

Name	Type	Source
ActionProposal: model_id, sequence, observation_ms, joint_velocity	str; int64 >= 0; int64; float64[n] rad/s	C2 over I4
RobotState: measured_ms, joint_position, joint_velocity	int64; float64[n] rad; float64[n] rad/s	C5 feedback over I7
Session: checkpoint_id, MotionLimits row, kinematic model, last_sequence, horizon T	loaded once by authorize_trial; last_sequence starts at -1; T in seconds	C4 memory
Output: CheckResult: decision, event_codes, command	enum PASS | LIMITED | REJECT | STOP; list[str]; CheckedCommand or null	to C5 over I7
CheckedCommand: sequence, joint_velocity, sent_ms, heartbeat, valid_for_ms	int64; float64[n] rad/s; int64; int64; int (50)	same fields as D0's I7 example

Figure 2. check_command steps. Source: Design_Diagrams/fig2_check_command.dot.



Procedure. The steps follow Figure 2 and the pseudocode below. Steps 1 to 4 check that the message is valid: a trial is running and the message comes from the approved model (step 1), it has one finite number per joint (step 2), its sequence number is higher than the last one accepted (step 3), and its data is no more than 50 ms old (step 4). If no trial is running or step 3 fails, C4 ignores that one message, and the previous command expires after 50 ms. If any other check fails, C4 stops the trial.

Steps 5 to 7 slow the command down when needed, but never speed it up. Step 5 caps each joint at its speed limit. Step 6 looks ahead by the robot's stopping time T and slows any joint that would otherwise reach the end of its range, minus a safety margin, within that time. A joint already past the margin may only stop or move back, and C4 never makes a joint move that the model did not ask to move. Step 7 uses the arm's geometry to work out how fast the hand would move; if that is over 0.25 m/s, every joint is slowed by the same fraction, so the hand keeps its direction. Step 8 predicts where the hand will be after time T and stops the trial if that point is outside the approved workspace box.

If steps 5 to 7 changed anything, the decision is LIMITED and the reason is recorded. Every step is logged in the background, so writing to disk never delays the robot.

```
function check_command(proposal, state, session, now_ms):
    limits = session.limits          # approved MotionLimits row
    T      = session.horizon_T       # stopping time in seconds
    m      = limits.margin           # safety margin in radians
```

```
    events = empty list

    # Steps 1-4: is the message valid?
    if no trial is running:                          return REJECT("E_NOT_RUNNING")
    if proposal.model_id != session.checkpoint_id:   return STOP("E_WRONG_MODEL")
    if length(proposal.joint_velocity) != n:         return STOP("E_DIM_MISMATCH")
    if any value is NaN or infinite:                 return STOP("E_NONFINITE")
    if proposal.sequence <= session.last_sequence:   return REJECT("E_REPLAY")
    if now_ms - proposal.observation_ms > 50:        return STOP("E_STALE_OBS")
    if now_ms - state.measured_ms > 50:              return STOP("E_STALE_FEEDBACK")

    v = copy of proposal.joint_velocity

    # Step 5: cap each joint's speed
    for each joint i:
        if abs(v[i]) > limits.v_max[i]:
            v[i] = sign(v[i]) * limits.v_max[i]
            add "W_JOINT_LIMITED" to events

    # Step 6: a joint must be able to stop before the end of its range
    for each joint i:
        room_up   = limits.q_max[i] - m - state.q[i]    # radians left going up
        room_down = state.q[i] - (limits.q_min[i] + m)  # radians left going down
        fastest_up   = max(0, room_up / T)       # never below 0: no invented motion
        fastest_down = max(0, room_down / T)
        if v[i] >  fastest_up:   v[i] =  fastest_up;   add "W_RANGE_LIMITED"
        if v[i] < -fastest_down: v[i] = -fastest_down; add "W_RANGE_LIMITED"

    # Step 7: cap hand speed
    hand_speed = length(jacobian(state.q) * v)   # m/s, Pinocchio
    if hand_speed > limits.ee_speed_max:         # 0.25 m/s
        v = v * (limits.ee_speed_max / hand_speed)
        add "W_EE_LIMITED" to events

    # Step 8: hand must stay inside the workspace
    future_hand = hand_position(state.q + v * T) # forward kinematics, Pinocchio
    if future_hand is outside limits.workspace_box: return STOP("E_WORKSPACE")

    session.last_sequence = proposal.sequence
    decision = LIMITED if events is not empty else PASS
    command  = CheckedCommand(proposal.sequence, v, now_ms,
                              session.heartbeat, valid_for_ms = 50)
    queue_log_row(proposal, v, decision, events)  # background, never waits
    return decision, events, command

# STOP sends zero velocity, ends the trial and logs the reason.
# REJECT ignores this message only; the last command expires after 50 ms.
```

Complexity at realistic size. Each step does a fixed amount of work per joint, so the cost grows only with the number of joints n (O(n)), not with how long the trial runs. That is a few hundred arithmetic operations, far below the 5 ms budget. The real risk is Python itself occasionally pausing, so we will time 10,000 test calls and check the slowest 1% before using hardware. At 100 times our data size, trials get longer but each step costs the same, so timing is unaffected; only the log file grows.

Why this algorithm rather than alternatives. We considered four alternatives.

- **Relying only on the robot controller's own joint limits.** Rejected, because the controller does not know about hand speed, our workspace, stale data or which model is running.

- **Rejecting every over-limit command instead of slowing it down.** Simpler, but learned models often go slightly over a limit, so trials would keep stopping before the 100-step target. We slow the command and log it instead.
- **An optimization-based safety filter.** It would find the smallest safe change and could avoid collisions, but it needs a solver running every cycle and collision models we do not have, and it is hard to prove fast enough. We list it as future work.

Edge cases.

Case	Behaviour	Tied to
Empty vector or length ≠ n	STOP, E_DIM_MISMATCH	I4, US-04
NaN or infinite value	STOP, E_NONFINITE	I4
model_id is not the authorized checkpoint	STOP, E_WRONG_MODEL	I4, I9
Duplicate or older sequence number (replay)	REJECT, E_REPLAY; previous command expires after 50 ms, then zero velocity	I7
Gap in sequence numbers	Accept, log W_SEQ_GAP	I4
Observation older than 50 ms / feedback older than 50 ms	STOP, E_STALE_OBS / E_STALE_FEEDBACK	I6, I7, timing budget
Value exactly at a limit; all-zero vector	Passes (comparisons use ≤); zero is always allowed and is the hold command	US-04
Joint already beyond its safety margin	Only zero or inward motion allowed (step 6)	US-04
Arm near a kinematic singularity	No Jacobian inverse is used, so the speed check stays well defined	US-03
First proposal of a trial	last_sequence = −1, so sequence 0 is accepted	I4
Proposal produced on another computer	Not supported: timestamps must share one monotonic clock	I4
Log queue full or write fails	STOP, E_LOG_FAILED (D0: incomplete logs mean incomplete tests)	US-04
C4 itself crashes	Heartbeat stops; the controller's independent timeout stops the robot	I8, D0 Sec. 3

3.2 authorize_trial (C4, before each physical trial)

What it is and the problem it solves. authorize_trial is the single place that enforces simulation evidence before hardware (US-04) and the pre-trial safety checks (US-03). If it is wrong, an untested or under-performing model reaches the real arm.

Inputs and outputs. Input is a TrialRequest over I9: checkpoint_id, robot_id, limits_id, eval_id (str), report_reviewed and estop_tested (bool), operator (str). It reads the registry rows in Figure 1 and the success threshold (float, 0.60, a draft target). Output is a TrialDecision: allowed (bool), trial_id (str or null), success_rate (float 0 to 1), episodes_completed (int) and reasons (list of error codes, empty when allowed).

Procedure. It evaluates every check and returns all failing reasons, not only the first, so the researcher can fix everything in one pass. The checks are: the checkpoint exists with status complete; the evaluation run exists, is complete and matches the requested checkpoint and robot; the run's completed episodes equal planned_episodes and planned_episodes is at least 50; the success rate recomputed from Episode rows meets the threshold; the report file's SHA-256 matches the stored hash, so it was not edited after review; the

limits row belongs to this robot, is approved and is the latest version; both booleans are true; and no other trial is running. If every check passes, it inserts a Trial row and loads the session state used by check_command.

Complexity at realistic size. Counting a run's episodes uses the Episode primary key: O(E) with E = 50, microseconds. At 100 times (E = 5,000) it is still under a millisecond. Hashing the report is O(file size), a few kilobytes. The cost that does matter is the model weights, which can be several gigabytes, so hashing them takes tens of seconds; the gate therefore compares file size and modification time against the registry, and C2 verifies the full hash when it loads the weights.

Why this algorithm rather than alternatives. A manual checklist alone was rejected because nothing enforces it. Trusting a success rate stored in the report was rejected because a hand edit or duplicated episode could pass. The open question is the statistical test. Comparing the point estimate with 60%, as the D0 target implies, passes 30 successes out of 50, yet the 95% Wilson interval for 30 of 50 is about 0.46 to 0.72, and a model whose true success rate is 50% would still pass about 10% of the time. Requiring the Wilson lower bound to clear the threshold is stricter but needs either more episodes or a lower threshold

Edge case	Behaviour
Zero completed episodes	Refuse with E_REPORT_INCOMPLETE; no division is attempted
Success rate exactly at the threshold	Passes (≥) x
Failed or incomplete episodes	Not counted as completed, so the run is incomplete and refused (D0 I3)
Run evaluated a different checkpoint or robot	Refuse with E_REPORT_MISMATCH
Report file changed after its hash was stored	Refuse with E_REPORT_TAMPERED
Newer limits version approved since the request was written	Refuse with E_LIMITS_NOT_CURRENT; resubmit with the new limits_id
A trial is already running	Refuse with E_TRIAL_ACTIVE (one active robot, D0 Sec. 6)
Previous trial ended in a fault	A fresh authorize_trial is required (D0 I9)

4 Build-versus-reuse decisions

Licenses below are for the projects as we understand them;

Piece	Build or reuse	Library or service	License	Reason
Pretrained world action model (C2)	Reuse	candidates: Octo; openpi (π0); OpenVLA	Octo MIT; openpi Apache-2.0; OpenVLA code MIT, weights Llama 2 Community License	Training from scratch is outside our compute (D0 decision 1). Choose by GPU memory, whether fine-tuning code is published, and action space.
Fine-tuning and inference code (C2)	Reuse, then adapt	The chosen model's own repository	Same as model	Its data loaders and action heads are tested; we change only configuration and the robot-specific head.
Deep-learning framework (C2)	Reuse	JAX (Octo, openpi) or PyTorch (OpenVLA)	Apache-2.0 / BSD-3-Clause	Follows the model; mixing frameworks adds work for no gain.

Piece	Build or reuse	Library or service	License	Reason
Experiment registry (C2)	Reuse engine, build schema	SQLite via Python sqlite3	Public domain / PSF	Mature, in standard library, no server (Section 2).
Dataset format (C1, C2)	Reuse	The model's format (RLDS or LeRobot)	Apache-2.0	Converting once at import avoids a custom loader.
Forward kinematics and Jacobian (C4)	Reuse	Pinocchio	BSD-2-Clause	Reads URDF directly; fast C++ core with Python bindings; widely used. Hand-written kinematics is an easy place to make silent errors.
check_command logic (C4)	Build	NumPy for vector maths	BSD-3-Clause	Project-specific and safety-critical; must be small enough to read and test line by line.
authorize_trial gate (C4)	Build	sqlite3	PSF	The rules are specific to our requirements.
Wilson interval (C4)	Reuse	statsmodels proportion_confint	BSD-3-Clause	Standard, tested implementation; only used if the lower-bound rule is chosen.
Message schemas and validation (C2, C4)	Reuse	pydantic v2	MIT	Typed models generate the error for a missing or mistyped field; compiled core is fast enough for 20 messages per second
Local transport (I4, I7, I9)	Reuse	Python socket (Unix domain sockets) and json	PSF	Already chosen in D0; no extra dependency.
Heartbeat timer and log writer (C4)	Build on standard library	threading, queue	PSF	A few dozen lines; behaviour must match D0's 20 ms heartbeat exactly.
Command-line tools (I1, I9)	Reuse	argparse	PSF	Keyboard-only text interface meets D0's accessibility note.
Tests, including property tests of the checker	Reuse	pytest, Hypothesis	MIT / MPL-2.0	Hypothesis generates random proposals to check that output never exceeds a limit.

5 API contract

All C4 methods are JSON messages over C4's Unix domain socket; register_checkpoint is a Python method of C2's registry module. Every message is a JSON object with `api_version` (str, "MAJOR.MINOR") and `type` (str). A failed request returns `"ok": false` and an `error` object with `code` (str), `message` (str, human readable) and `action` (one of refuse, reject, stop). Callers must treat any unknown error code as stop.

5.1 authorize_trial (I9, researcher CLI to C4)

Parameter	Type	Required	Valid range
checkpoint_id, robot_id, limits_id, eval_id	str	Yes	Must exist in the registry

Parameter	Type	Required	Valid range
report_reviewed, estop_tested	bool	Yes	Must be true to be allowed
operator	str	Yes	1 to 64 characters
note	str	No	Up to 500 characters; stored on the Trial row

Output: `{ok: bool, allowed: bool, trial_id: str | null, success_rate: float 0–1, episodes_completed: int, reasons: list[str]}`.

5.2 check_command (I4 in from C2, I7 out to C5)

Parameter	Type	Required	Valid range
model_id	str	Yes	Equals the authorized checkpoint_id
sequence	int64	Yes	Greater than the last accepted value
observation_ms	int64 (ms, monotonic)	Yes	now − observation_ms ≤ 50
joint_velocity	float64[n] (rad/s)	Yes	Length n, all finite; larger values are limited, not refused

Output: `{ok: bool, decision: "PASS"|"LIMITED"|"REJECT"|"STOP", event_codes: list[str], command: {sequence: int64, joint_velocity: float64[n] rad/s, sent_ms: int64 ms, heartbeat: int64, valid_for_ms: int ms} | null}`. command is null for REJECT and STOP.

5.3 stop_trial (I9)

Inputs: trial_id (str, required), reason (str, optional, up to 200 characters). Output: `{ok: true, status: "stopped", ended_ms: int64}`. Stopping always sends zero velocity first and is idempotent: stopping an already-stopped trial returns E_NO_TRIAL with action none.

5.4 register_checkpoint (C2 registry, used when C2 finishes adaptation over I2)

Inputs: weights_path (str, required, existing file), dataset_ids (list[str], required, at least one, all registered), base_model (str, required), code_version (str, required, 40-character git commit hash with no uncommitted changes), settings (object, required, saved as settings_json). Output: `{ok: true, checkpoint_id: str, weights_sha256: str (64 hex), status: "complete"}`. A checkpoint whose training did not finish is never registered as complete (D0 I2).

Error responses

Code	Method	When it happens	Caller action
E_API_VERSION	any	Major version differs from C4's	Refuse
E_NOT_RUNNING	check_command	No authorized trial is active	Reject
E_WRONG_MODEL	check_command	model_id ≠ authorized checkpoint	Stop
E_DIM_MISMATCH / E_NONFINITE	check_command	Length ≠ n, or NaN/Inf value	Stop
E_REPLAY	check_command	sequence ≤ last accepted	Reject
E_STALE_OBS / E_STALE_FEEDBACK	check_command	Data older than 50 ms	Stop
E_WORKSPACE	check_command	Predicted position outside box	Stop
E_LOG_FAILED	check_command	Log queue full or write error	Stop

Code	Method	When it happens	Caller action
W_JOINT_LIMITED / W_RANGE_LIMITED / W_EE_LIMITED / W_SEQ_GAP	check_command	Steps 5, 6, 7 reduced speed; or sequence gap (warnings, decision LIMITED or PASS)	Continue
E_CHECKPOINT_PARTIAL, E_REPORT_MISSING, E_REPORT_INCOMPLETE, E_REPORT_MISMATCH, E_REPORT_TAMPERED, E_BELOW_THRESHOLD	authorize_trial	The corresponding check in Section 3.2 fails	Refuse
E_LIMITS_NOT_APPROVED / E_LIMITS_NOT_CURRENT, E_REVIEW_MISSING, E_ESTOP_UNTESTED, E_TRIAL_ACTIVE	authorize_trial	Limits, booleans or active-trial check fails	Refuse
E_NO_TRIAL	stop_trial	trial_id unknown or already ended	None (stop is idempotent)
E_FILE_MISSING, E_DATASET_UNKNOWN, E_DUPLICATE_WEIGHTS, E_DIRTY_CODE	register_checkpoint	Weights file absent; dataset not in registry; same hash already registered; code_version has uncommitted changes	Refuse

Example request and response (check_command, two-joint test fixture from D0)

```
// C2 -> C4 over I4
{"api_version": "1.0", "type": "action_proposal",
 "model_id": "ckpt_0007", "sequence": 12, "observation_ms": 987,
 "joint_velocity": [0.02, -0.01]}

// C4 -> C5 over I7 (proposal within limits)
{"api_version": "1.0", "type": "checked_command", "ok": true,
 "decision": "PASS", "event_codes": [],
 "command": {"sequence": 12, "joint_velocity": [0.02, -0.01],
             "sent_ms": 1000, "heartbeat": 61, "valid_for_ms": 50}}

// Same proposal arriving 63 ms after its observation
{"api_version": "1.0", "type": "checked_command", "ok": false,
 "decision": "STOP", "event_codes": ["E_STALE_OBS"], "command": null,
 "error": {"code": "E_STALE_OBS", "action": "stop",
           "message": "observation is 63 ms old (limit 50 ms)"}}
```

Versioning. Every message carries `api_version` as MAJOR.MINOR and C4 refuses a request whose MAJOR differs from its own, while the registry records its schema version in SQLite's `PRAGMA user_version` and changes only through numbered migration scripts; a breaking change, which needs a new MAJOR, is removing or renaming a field, changing a field's type, unit or joint order, or making C4 accept an input it used to refuse, whereas adding an optional field or a new error code is a MINOR change.

6 Technology choices with justification

Backend language: Python 3.11 or later. *Team skill:* we have used many Python machine-learning libraries and frameworks in personal projects and hackathons. *Licensing:* Python's license is permissive, and every library in Section 4 works with Python. *Community:* Python is the main language of robot-learning research,

so the models, Pinocchio, MuJoCo and robot drivers all come with Python examples. *Performance:* the heavy maths runs in fast compiled libraries; Python's own overhead is the only risk, and we will measure it against the 5 ms budget. *Cost and hosting:* free, and already installed on lab and UC machines. *Rejected:* C++ for C4. It has more predictable timing, but two languages is too much for a two-person team. We will switch only if Python is measured too slow.

Machine-learning framework: PyTorch. *Team skill:* The team is adequately skilled with these tools. *Licensing:* both are free, open-source licenses (JAX: Apache-2.0, PyTorch: BSD-3-Clause). *Community:* both are widely used; we pick the one the chosen model's official code uses. *Performance:* both are fast on a GPU; prediction must fit the 30 ms budget, which we will measure. *Cost and hosting:* free, and both run on UC GPUs. *Rejected:* the other framework, because converting a pretrained model between frameworks is a project in itself.

Simulator: MuJoCo. *Licensing:* free and open source (Apache-2.0). *Community:* maintained by Google DeepMind, widely used, with ready-made robot models. *Performance:* runs fast on a normal CPU, so many tests can run at once without using the model's GPU. *Cost and hosting:* free on any lab or UC machine. *Rejected:* NVIDIA Isaac Sim, which has more realistic images but needs a high-end GPU and has a proprietary license; and PyBullet, which is no longer actively developed. [TEAM: if realistic camera images matter a lot for your model, check this choice with your advisor.]

Database: SQLite plus files. *Team skill:* Taken college courses and built applications; we use plain SQL, so there is little to learn. *Licensing:* public domain. *Community:* one of the most widely used databases, well documented, and built into Python. *Performance:* our busiest table gets only 20 new rows per second, which is easy for SQLite, and logging happens in the background. *Cost and hosting:* just a file in our private project storage, with no server to run. *Rejected:* PostgreSQL. It is built for many users writing at once, but we have one machine and one trial, and running a database server goes against D0's "no separate storage service" decision.

Messaging and hardware interface: JSON over local sockets; the robot's own interface inside C5. *Licensing:* sockets and JSON are built into Python; the robot interface is manufacturer SDK. *Community:* sockets and JSON are universal; ROS 2 has the largest robotics community. *Performance:* local messages take well under 1 ms, inside the 5 ms budget. *Cost and hosting:* free and local, with no network needed. *Rejected:* ROS 2 for messages between our components. It has good tools and robot drivers, but it is a lot to learn on top of the model and simulator. If our robot's driver needs ROS 2, C5 will handle it, and the rest of the system stays the same.

Compute and hosting: training on UC research computing cluster or lab GPU, the model, C4 and C5 run on one lab computer connected to the robot. *Licensing:* university resources under UC's usage rules. *Community:* support from UC's research computing staff and documentation. *Performance:* running on one computer means C2 and C4 share the same clock, and training never slows down the live robot's GPU. *Cost and hosting:* no cloud bills, and camera data stays on university storage, which protects privacy (US-05). *Rejected:* rented cloud GPUs, because they cost money we haven't budgeted and would send camera data off campus. There is no graphical interface; command-line tools keep D0's keyboard-and-text accessibility approach.